

PostgreSQL Learning Plan

A structured, day-wise roadmap from Beginner to Advanced

30-day plan · ~1–2 hours per day · hands-on focused

How to use this plan: Each day has a focused topic, key concepts, and a hands-on task. Spend roughly 60–70% of your time practicing in `psql` or pgAdmin, not just reading. The plan is split into three phases: **Beginner** **Intermediate** **Advanced**. If a day feels heavy, split it across two days — consistency matters more than speed.

Phase 1 — Foundations (Days 1–10)

Beginner

Day	Topic	What to learn	Hands-on task
Day 1	Setup & Orientation	Install PostgreSQL, pgAdmin, and <code>psql</code> . Server vs database vs schema. Connecting.	Install PostgreSQL, connect via <code>psql</code> , run <code>SELECT version();</code> , create your first database.
Day 2	psql Basics	Meta-commands: <code>\l \c \dt \d \du \q</code> . Running SQL files. Getting help.	Create a <code>learning</code> DB, list tables, explore <code>\?</code> and <code>\h</code> .
Day 3	Data Types	<code>integer, numeric, varchar, text, boolean, date, timestamp, serial</code> . When to use each.	Create a <code>customers</code> table using at least 6 different data types.
Day 4	CREATE TABLE & Constraints	PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, CHECK, FOREIGN KEY.	Design <code>customers</code> + <code>orders</code> with a foreign key and check constraints.
Day 5	INSERT / UPDATE / DELETE	Single & multi-row inserts, <code>RETURNING</code> , safe updates/deletes with WHERE.	Insert 10 customers, update some, delete one, use <code>RETURNING *</code> .
Day 6	SELECT & Filtering	<code>WHERE</code> , comparison & logical operators, <code>BETWEEN, IN, LIKE, ILIKE, IS NULL</code> .	Write 10 queries filtering the customers/orders data various ways.
Day 7	Sorting & Limiting	<code>ORDER BY</code> , <code>LIMIT / OFFSET</code> , <code>DISTINCT</code> , <code>FETCH FIRST</code> .	Top-5 orders by amount; paginate results with LIMIT/OFFSET.
Day 8	Aggregates & GROUP BY	<code>COUNT, SUM, AVG, MIN, MAX</code> , <code>GROUP BY</code> , <code>HAVING</code> .	Total & average order amount per customer; filter groups with HAVING.
Day 9	Joins	INNER, LEFT, RIGHT, FULL, CROSS joins. Join conditions & aliases.	Join customers & orders; find customers with no orders (LEFT JOIN + IS NULL).
Day 10	Review + Mini Project	Consolidate Days 1–9. Basic schema design.	Build a small "library" or "inventory" schema and load sample data end-to-end.

Phase 2 — Intermediate (Days 11–20)

Intermediate

Day	Topic	What to learn	Hands-on task
-----	-------	---------------	---------------

Day 11	Subqueries	Scalar, correlated subqueries, <code>IN / EXISTS / NOT EXISTS</code> .	Find customers whose order total exceeds the average using a subquery.
Day 12	CTEs (WITH)	Common Table Expressions, chaining CTEs, readability vs subqueries.	Rewrite a nested subquery as a multi-step <code>WITH</code> chain.
Day 13	CASE & Conditional Logic	<code>CASE WHEN</code> , <code>COALESCE</code> , <code>NULLIF</code> , <code>GREATEST/LEAST</code> .	Bucket orders into Low/Medium/High tiers with CASE.
Day 14	String & Date Functions	<code>concat</code> , <code>substring</code> , <code>replace</code> , <code>split_part</code> ; <code>now()</code> , <code>age()</code> , <code>date_trunc</code> , <code>to_char</code> .	Format dates, extract month, clean text fields.
Day 15	Window Functions	<code>ROW_NUMBER</code> , <code>RANK</code> , <code>DENSE_RANK</code> , <code>LAG</code> , <code>LEAD</code> , <code>OVER(PARTITION BY ...)</code> .	Rank orders per customer; running total of order amounts.
Day 16	Set Operations	<code>UNION / UNION ALL</code> , <code>INTERSECT</code> , <code>EXCEPT</code> .	Combine two query results; find rows in A not in B with EXCEPT.
Day 17	Indexes	B-tree, unique, composite, partial indexes. When indexes help/hurt.	Create indexes and compare query plans before/after.
Day 18	Views & Materialized Views	<code>CREATE VIEW</code> , updatable views, <code>MATERIALIZED VIEW</code> + <code>REFRESH</code> .	Build a reporting view; create and refresh a materialized view.
Day 19	Transactions	<code>BEGIN/COMMIT/ROLLBACK</code> , SAVEPOINT, ACID, isolation levels overview.	Simulate a money-transfer transaction with rollback on error.
Day 20	Review + Reporting Project	Consolidate intermediate topics.	Build a sales-reporting query set using CTEs + window functions + views.

Phase 3 — Advanced (Days 21–30)

Advanced

Day	Topic	What to learn	Hands-on task
Day 21	JSON & JSONB	<code>json</code> vs <code>jsonb</code> , operators <code>-></code> <code>->></code> <code>#></code> , <code>jsonb_set</code> , GIN indexes.	Store & query a JSONB column; index it and query nested keys.
Day 22	Arrays & Advanced Types	Array columns, <code>unnest</code> , <code>ANY/ALL</code> , <code>hstore</code> , ranges, <code>UUID</code> .	Model tags as an array; query with <code>ANY</code> and <code>unnest</code> .
Day 23	Functions (PL/pgSQL)	<code>CREATE FUNCTION</code> , parameters, <code>RETURNS</code> , variables, <code>IF/LOOP</code> .	Write a function that returns a customer's order summary.
Day 24	Stored Procedures & Triggers	<code>CREATE PROCEDURE</code> , <code>CALL</code> ; triggers & trigger functions (<code>BEFORE/AFTER</code>).	Add an audit trigger that logs every UPDATE on orders.
Day 25	Query Performance	<code>EXPLAIN</code> , <code>EXPLAIN ANALYZE</code> , reading plans, seq scan vs index scan.	Analyze a slow query, add an index, and measure the improvement.
Day 26	Advanced Indexing & Optimization	GIN/GiST/BRIN, covering indexes, <code>VACUUM</code> , <code>ANALYZE</code> , statistics.	Choose the right index type for full-text/JSONB and verify with EXPLAIN.

Day 27	Partitioning	Range/list/hash partitioning, partition pruning, when to partition.	Create a range-partitioned <code>orders</code> table by order date.
Day 28	Security & Roles	Roles, <code>GRANT</code> / <code>REVOKE</code> , schemas, Row-Level Security (RLS).	Create read-only and app roles; enable RLS on a table.
Day 29	Backup, Restore & Maintenance	<code>pg_dump</code> , <code>pg_restore</code> , <code>psql</code> restore, WAL & replication overview.	Dump and restore your learning database to a fresh DB.
Day 30	Capstone Project	Bring everything together: schema design, functions, triggers, indexing, reporting.	Build a complete mini-app schema (e-commerce or ticketing) with reports, functions, triggers, and tuned queries.

Practice Tips

- **Type everything.** Don't copy-paste — muscle memory for SQL matters.
- **Use a real dataset.** Load the free `pagila` or `dvdrental` sample DB and query it daily.
- **Read query plans early.** Get comfortable with `EXPLAIN ANALYZE` from Phase 2 onward.
- **Keep a snippets file.** Save useful queries and function templates as you go.
- **Break & fix.** Deliberately cause errors (bad FK, constraint violation) to learn the messages.

Recommended Resources

- **Official docs:** [postgresql.org/docs](https://www.postgresql.org/docs/) — the tutorial + SQL reference are excellent.
- **Practice:** pgexercises.com, [LeetCode](https://leetcode.com/)/[HackerRank](https://www.hackerrank.com/) SQL, [DB Fiddle](https://dbfiddle.uk/) for quick tests.
- **Sample DBs:** Pagila, DVD Rental, Chinook.
- **Tools:** pgAdmin, DBeaver, and the `psql` CLI.